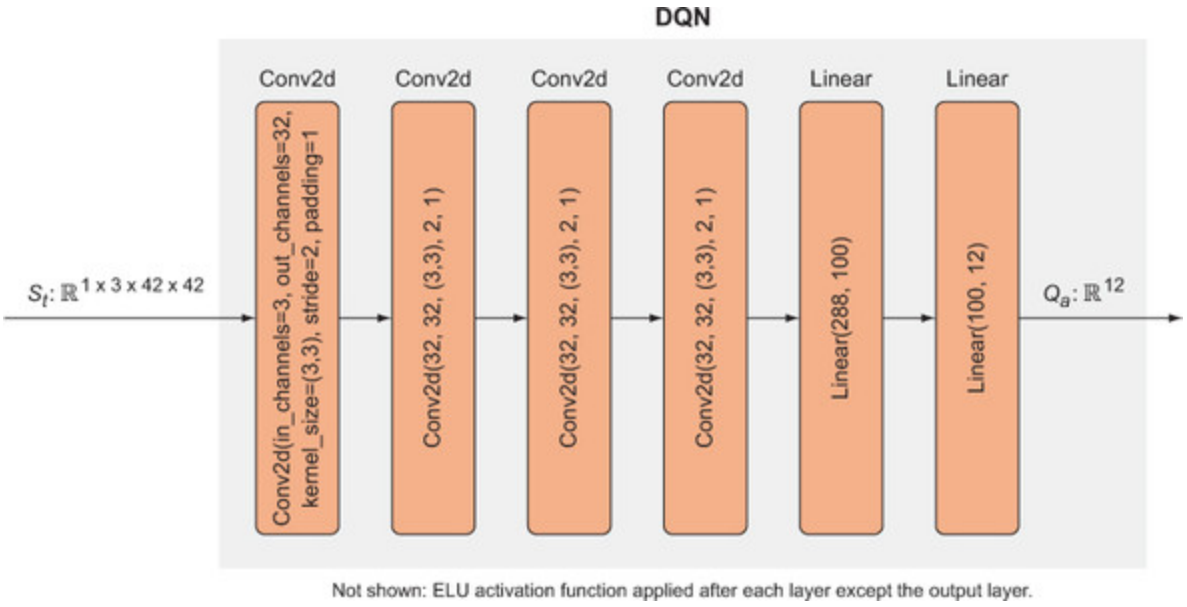


Our DQN will learn to predict the expected rewards for each possible action given the state (i.e., action values or Q values), and we use these action values to decide which action to take. Naively we should just take the action associated with the highest value, but our DQN will not produce accurate action values in the beginning, so we need to have a policy that allows for some exploration so the DQN can learn better action-value estimates.

Earlier we discussed using the epsilon-greedy policy, where we take a random action with probability ε and take the action with the highest value with probability (1 – ε). We usually set ε to be some reasonably small probability like 0.1, and often we'll slowly decrease ε during training so that it becomes more and more likely to choose the highest value action.

We also discussed sampling from a softmax function as our policy. The softmax function essentially takes a vector input with arbitrary real numbers and outputs a same-sized vector where each element is a probability, so all elements sum to 1. It therefore creates a discrete probability distribution. If the input vector is a set of action values, the softmax function will return a discrete probability distribution over the actions based on their action values, such that the action with the highest action value will be assigned the highest probability. If we sample from this distribution, the actions with the highest values will be chosen more often, but other actions will also be